Architecture Big Data appliquée à l'IA

Séance 1 : architecture d'un data warehouse

Introduction : le cours Architecture Big Data appliquée à l'IA

Le but du cours : comprendre comment on stocke, traite et exploite de très gros volumes de données dans le cloud, et savoir **choisir et justifier une architecture**.

Format : 5 séances, chacune avec **3 h de cours** puis **4 h de TP** sur des offres gratuites (Google Cloud et BigQuery, Databricks Free Edition).

Le projet conducteur : un site e-commerce fictif qui veut **détecter la fraude au paiement**. Ses données (clients, commandes, paiements, navigation sur le site) servent dans toutes les séances : chaque TP construit une brique du même système.

Introduction : le cours Architecture Big Data appliquée à l'IA (suite)

Séance	Thème	Ce que vous apprendrez	Briques du TP
1	**Data warehouse**	Le matériel derrière le cloud, le stockage en colonnes, le traitement en parallèle et ce qui fait le coût d'une requête	Modéliser un entrepôt en étoile, le charger sur BigQuery et Databricks, mesurer l'effet du partitionnement et du clustering
2	**Lakehouse**	Les limites du couple data lake + data warehouse, les formats de table ouverts et l'organisation des données en couches	Construire les couches bronze, silver et gold des données du projet sur Databricks
3	**Traitement distribué**	Quand une seule machine suffit et quand il faut un cluster, et les principaux moteurs de calcul (Python, Polars, Ray, Spark)	Calculer des indicateurs de fraude avec plusieurs moteurs et comparer leurs performances
4	**Streaming avec Kafka**	Le traitement des données en continu, le modèle du journal distribué et les garanties de livraison	Monter un pipeline temps réel qui consomme des flux de données
5	**Déploiement ML / IA**	Le passage d'un modèle en production et ce qui rend un système de ML fiable	Déployer un modèle de détection de fraude qui note chaque paiement en temps réel et publie des alertes

Séance 1 : architecture d'un data warehouse

Objectif : comprendre **sur quel matériel** tourne un data warehouse cloud, et **pourquoi son architecture** (stockage colonnaire, séparation stockage/calcul, exécution massivement parallèle) découle directement des caractéristiques de ce matériel.

1. Du serveur au data center



Une allée de baies (racks) dans un data center : chaque baie empile des dizaines de serveurs.

1. Du serveur au data center (suite)

Points à faire passer :

- **Un serveur** est un ordinateur sans écran, optimisé pour tourner en continu : plusieurs processeurs, beaucoup de RAM, des disques, une ou plusieurs cartes réseau.
- **Une baie (rack)** regroupe environ 40 serveurs et un commutateur réseau (*top-of-rack switch*). Les baies sont reliées entre elles par un réseau de data center hiérarchique.
- **Un data center** contient des milliers à des dizaines de milliers de serveurs, plus l'alimentation électrique, le refroidissement et la sécurité physique. Une **région** cloud (ex. `europe-west1`) regroupe plusieurs data centers ou **zones** indépendantes.
- Dans le cloud, on ne loue pas une machine physique entière mais une **machine virtuelle** : un morceau de serveur (des vCPU, de la RAM) isolé.

1. Du serveur au data center (suite)

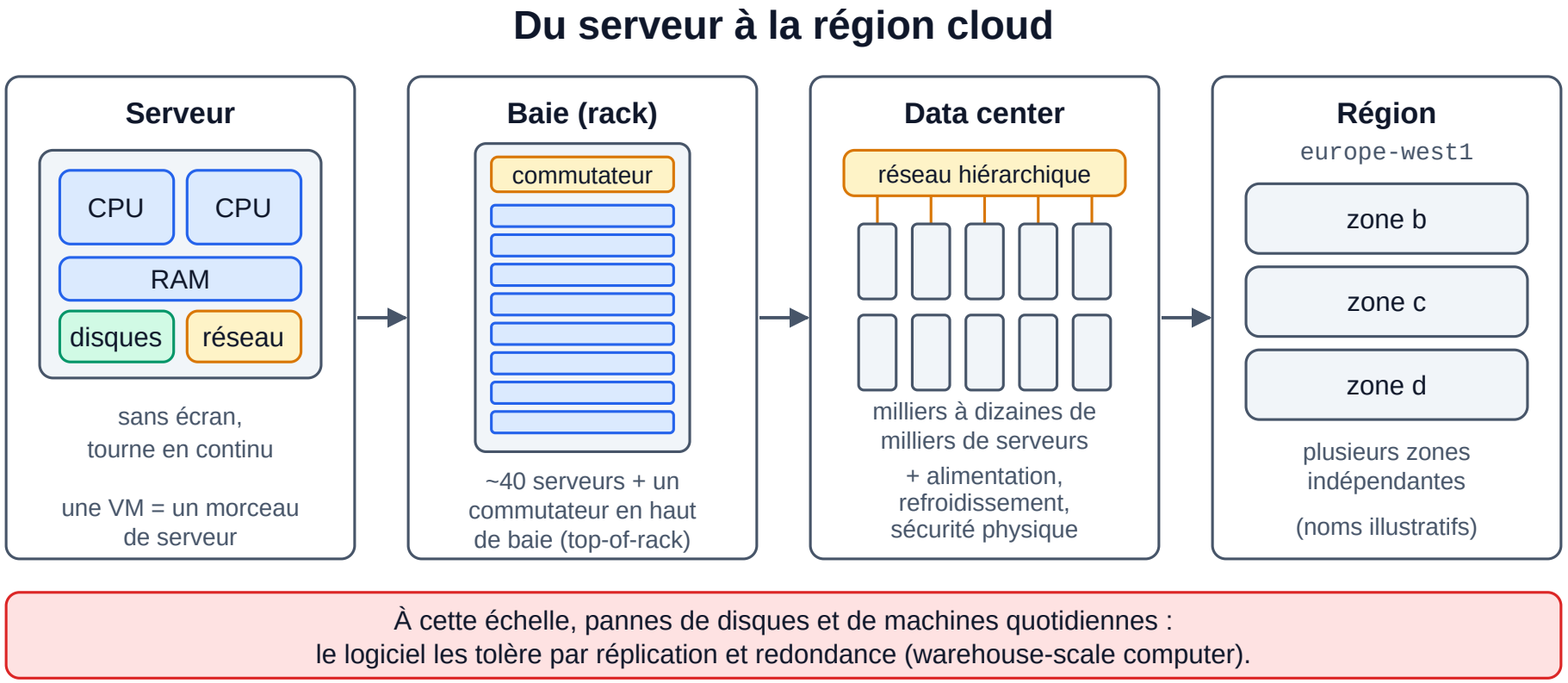


Schéma : chaque niveau contient le précédent ; à l'échelle de la région, les pannes deviennent quotidiennes et c'est le logiciel qui les absorbe.

Sources : 1. Du serveur au data center

Sources :

- Google Cloud, régions et zones.
- Google Cloud, plateformes CPU de Compute Engine (définition du vCPU).

2. Les composants d'une machine

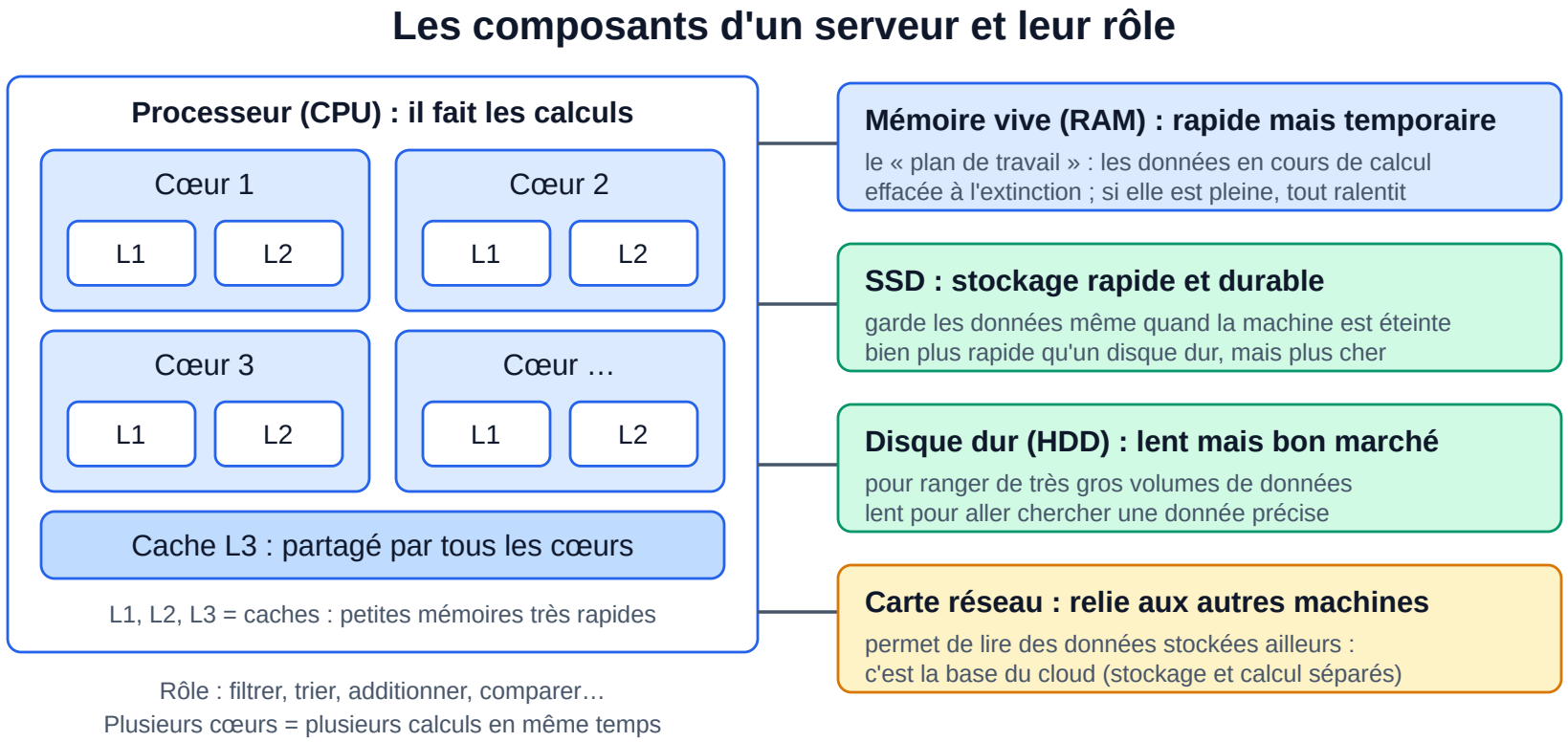


Schéma : le processeur calcule, la RAM garde les données en cours de travail, les disques les conservent, la carte réseau relie aux autres machines.

2.1 Le CPU (processeur) : là où le calcul se fait

- Le CPU exécute des instructions (additionner, comparer, copier). Sa vitesse dépend de la **fréquence** (en GHz, des milliards de cycles par seconde), du **nombre d'instructions par cycle** et du **nombre de cœurs**.
- Un **cœur** est une unité d'exécution indépendante. Un serveur cloud récent en possède des dizaines, voire plus d'une centaine.
- Chaque cœur dispose de **caches** (L1, L2, puis L3 partagé) : de petites mémoires très rapides intégrées à la puce, de quelques dizaines de Ko (L1) à quelques dizaines de Mo (L3). Le CPU y garde les données qu'il vient d'utiliser.
- **Dans un data warehouse**, le CPU décompresse, filtre, calcule les agrégats et les jointures. Une requête analytique est souvent limitée non par le CPU mais par la vitesse à laquelle on lui amène les données.

2.2 La RAM (mémoire vive) : l'espace de travail

- La RAM contient les données et programmes en cours d'utilisation. Elle est **volatile** : son contenu disparaît à l'extinction.
- Elle est environ **200 fois plus lente que le cache L1**, mais **1 000 fois plus rapide qu'un SSD** pour un accès aléatoire.
- **On ne peut pas tout garder en RAM**. On y met seulement les données en cours de calcul, et on stocke le reste sur SSD ou sur disque dur.
- **Dans un data warehouse**, la RAM sert aux tables de hachage des jointures et des `GROUP BY`, aux tris, et aux échanges de données entre étapes (*shuffle*). Quand elle ne suffit pas, le moteur **déborde sur disque** (*spill*), ce qui ralentit fortement la requête.

2.3 Moyens de stockage

Un moyen de stockage sert à **ranger les données pour les garder dans la durée**, même quand la machine est éteinte. C'est la différence avec la RAM, qui ne sert qu'à travailler sur les données du moment.

Dans le cloud : disque local ou disque réseau. Quand on loue une machine virtuelle, son stockage peut être placé à deux endroits.

	Disque local (*Local SSD* sur Google Cloud)	**Disque réseau (*Persistent Disk* sur Google Cloud)**
Où est-il ?	**Dans le serveur physique** qui fait tourner la machine virtuelle	**Ailleurs dans le data center**, relié à la machine par le réseau
Technologie	Toujours un SSD	Au choix : SSD (types *pd-balanced*, *pd-ssd*) ou disque dur (type *pd-standard*)
Vitesse	La plus rapide	Rapide, un peu moins (les données passent par le réseau)
Si la machine virtuelle est arrêtée ou supprimée	Les données sont **perdues**	Les données **sont conservées** : on peut rebrancher le disque sur une autre machine
À utiliser pour	Des données **temporaires** : cache, fichiers de travail	Des données **à garder**

2.3 Moyens de stockage (suite)

Pourquoi un disque local « perd » ses données alors qu'un SSD les garde sans courant ? Le SSD lui-même ne s'efface pas. Mais il est fixé dans un serveur physique précis, et la machine virtuelle ne fait que l'**emprunter**. Quand on arrête ou supprime la machine virtuelle, elle **libère ce serveur** ; au redémarrage, elle peut être placée sur un autre serveur, avec un autre SSD vide. Les données ne suivent pas la machine virtuelle. Le disque réseau, lui, n'appartient à aucun serveur : il existe indépendamment de la machine et on le rebranche où l'on veut.

2.3.1 Le SSD

On utilise le SSD pour les données qu'il faut **retrouver vite** : le système d'exploitation, les bases de données, les données consultées souvent.

Dans certains data warehouse, les SSD locaux servent de **cache** : ils gardent une copie des données souvent lues (« chaudes ») au plus près du calcul, tandis que l'original reste sur un stockage durable (par exemple Redshift).

2.3.2 Le disque dur (HDD)

- Dans un disque dur, les données sont **rangées les unes à la suite des autres**, chacune à un emplacement précis, comme les pages d'un livre. Pour lire une donnée, le disque doit d'abord **se rendre à son emplacement** (environ 8 ms), puis il lit **tout ce qui suit** à bonne vitesse.
- Conséquence : **lire des données rangées côte à côte est rapide, sauter d'un emplacement à l'autre est lent**. C'est pourquoi un data warehouse **range ensemble les données qui seront lues ensemble** (stockage en colonnes, section 4).
- Son atout : c'est le stockage **le moins cher au téraoctet**, idéal pour garder de très gros volumes. Les grands stockages du cloud (chez Google ou S3 chez AWS) reposent encore en grande partie sur des disques durs.

2.3.3 SSD ou disque dur : ce qui les différencie

Le SSD et le disque dur sont deux **mémoires persistantes** : elles gardent les données quand la machine est éteinte. La différence se fait au niveau de **quatre caractéristiques** :

Caractéristique	Disque dur (HDD)	SSD
Latence : temps pour obtenir une donnée précise	~8 ms	~0,1 ms
Accès aléatoire : combien de petites données (4 Ko), rangées à des endroits différents, le disque peut lire **en une seconde**	~100 (il doit se déplacer à chaque fois : 1 s ÷ 8 ms)	plus d'un million (il n'a rien à déplacer et traite beaucoup de demandes en même temps)
Débit séquentiel : lecture d'un gros bloc d'un seul trait	~50 Mo/s	~7 Go/s
Coût au téraoctet	le plus bas	environ 4 fois plus cher

Exemple pour l'accès aléatoire : afficher **10 000 commandes de clients différents**, rangées à 10 000 endroits du disque. Le disque dur, à environ 100 lectures par seconde, met **environ 100 secondes**. Le SSD, à plus d'un million de lectures par seconde, met **moins d'un centième de seconde**. Ce chiffre s'appelle les **IOPS** (*opérations d'entrée-sortie par seconde*) ; c'est l'indicateur que les fabricants et les clouds affichent pour les disques.

2.3.3 SSD ou disque dur : ce qui les différencie (suite)

Ce qui fait vraiment la différence :

- **En accès aléatoire, l'écart est énorme** : plusieurs milliers de fois. Le disque dur perd son temps à « aller chercher » chaque donnée.
- **En lecture séquentielle, l'écart est bien plus faible** : une fois au bon endroit, le disque dur lit les données à la suite à un débit correct.
- Donc : **le disque dur convient quand on lit beaucoup de données d'un seul trait, le SSD quand on lit beaucoup de petites données dispersées** ou quand il faut répondre vite.

Pourquoi on choisit l'un ou l'autre **dans un système informatique** :

- **SSD** : système d'exploitation, bases de données transactionnelles (beaucoup de petites lectures et écritures, par exemple une commande ou un paiement à la fois), caches.
- **Disque dur** : sauvegardes, archives, très gros volumes rarement relus, là où le prix au To compte plus que la vitesse.

2.3.3 SSD ou disque dur : ce qui les différencie (suite)

Et **dans un data warehouse** :

- Les requêtes analytiques lisent **de grandes quantités de données d'un seul trait** (des colonnes entières) : c'est une lecture séquentielle. Le gros des données peut donc rester sur un **stockage de masse bon marché** (le stockage distribué de Google ou S3 chez AWS, qui combinent disques et flash).
- Les données **souvent lues** (« chaudes ») sont copiées dans un **cache sur SSD** au plus près du calcul : c'est le cas par exemple de Redshift.
- Le format en colonnes et les gros blocs (bloc 4) servent justement à **transformer des lectures dispersées en lectures séquentielles**, pour tirer le meilleur de chaque type de stockage.

À retenir : **le disque dur pour stocker beaucoup pour pas cher, le SSD pour accéder vite à n'importe quelle donnée**. Un data warehouse combine les deux.

2.3.3 SSD ou disque dur : ce qui les différencie (suite)

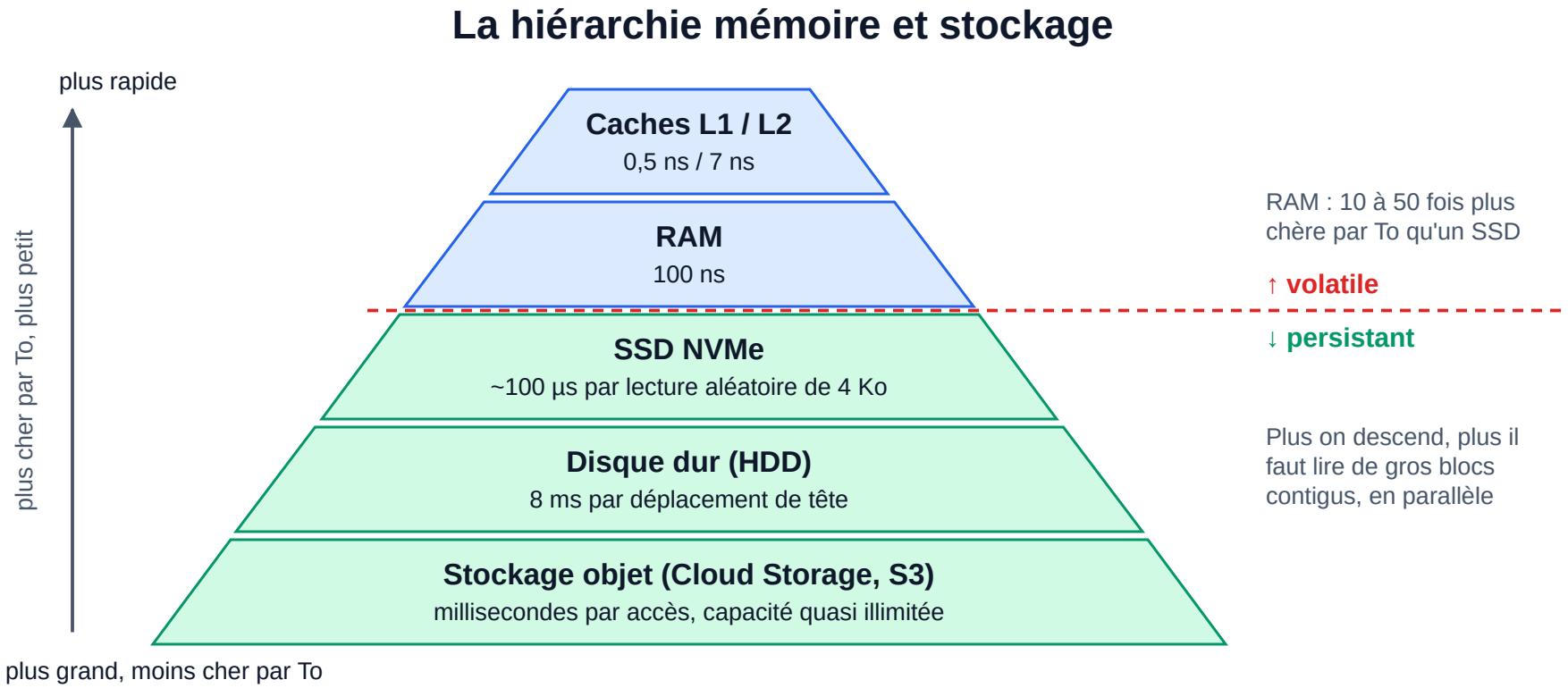


Schéma : en descendant, chaque niveau est plus lent mais plus grand et moins cher par To ; la ligne rouge sépare la mémoire volatile du stockage persistant.

2.4 Le réseau : ce qui relie tout

- **C'est ce qui rend possible la séparation du stockage et du calcul** : si le réseau est aussi rapide que le disque, il n'est plus nécessaire de stocker les données sur la machine qui calcule.
- Dans un data center moderne, le réseau est si rapide qu'il devient **comparable au disque local**. Le réseau de Google offre environ **1 Pbit/s de bande passante**, ce qui permet à 100 000 machines de communiquer entre elles à 10 Gbit/s chacune.

2.5 Ordres de grandeur à retenir

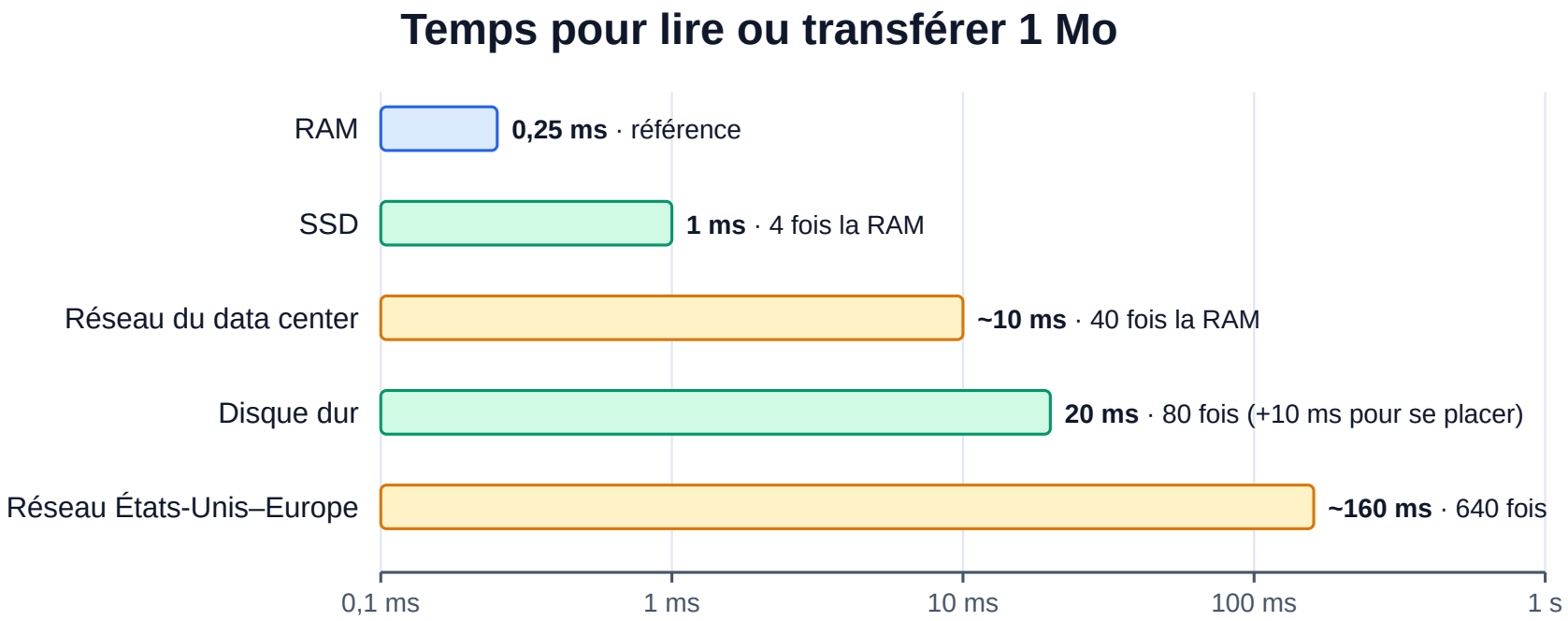
Mémoire	Temps pour lire ou transférer 1 Mo	Comparé à la RAM
Cache L1 / L2 du processeur	trop petits pour contenir 1 Mo (quelques dizaines de Ko à quelques Mo)	encore plus rapide
RAM	0,25 ms	1 (référence)
SSD	1 ms	4 fois plus lent
Autre machine du même data center	~10 ms	40 fois plus lent
Disque dur	20 ms	80 à 120 fois plus lent
Autre continent (aller-retour États-Unis–Europe)	~160 ms (150 ms de trajet + le transfert)	environ 640 fois plus lent

2.5 Ordres de grandeur à retenir (suite)

Ce qu'il faut en retenir :

- Pour une **grosse lecture**, les écarts restent raisonnables : le SSD est 4 fois plus lent que la RAM, le disque dur environ 100 fois.
- Pour des **petites lectures dispersées**, les écarts explosent (voir 2.3.3) : chaque lecture paie le temps d'accès, par exemple ~10 ms pour un disque dur qui doit se placer.
- Ces valeurs datent d'environ 2012. Le matériel actuel est plus rapide : un SSD lit 1 Mo en environ 0,15 ms, une carte réseau à 10 Gbit/s le transfère en moins de 1 ms (voir l'exercice). **Les écarts entre stockage et réseau se sont resserrés**, et c'est justement ce qui permet de séparer le stockage et le calcul (2.4).

2.5 Ordres de grandeur à retenir (suite)



Même quantité de données pour toutes les lignes (1 Mo, environ une photo). Chaque graduation multiplie le temps par 10.
Les caches L1 et L2, encore plus rapides, sont trop petits pour contenir 1 Mo. Valeurs : J. Dean et P. Norvig (~2012).

Schéma : les mêmes valeurs que le tableau, pour la même quantité de données (1 Mo) ; chaque graduation multiplie le temps par 10.

2.5 Ordres de grandeur à retenir (suite)

Exercice : combien de temps pour lire 1 To ?

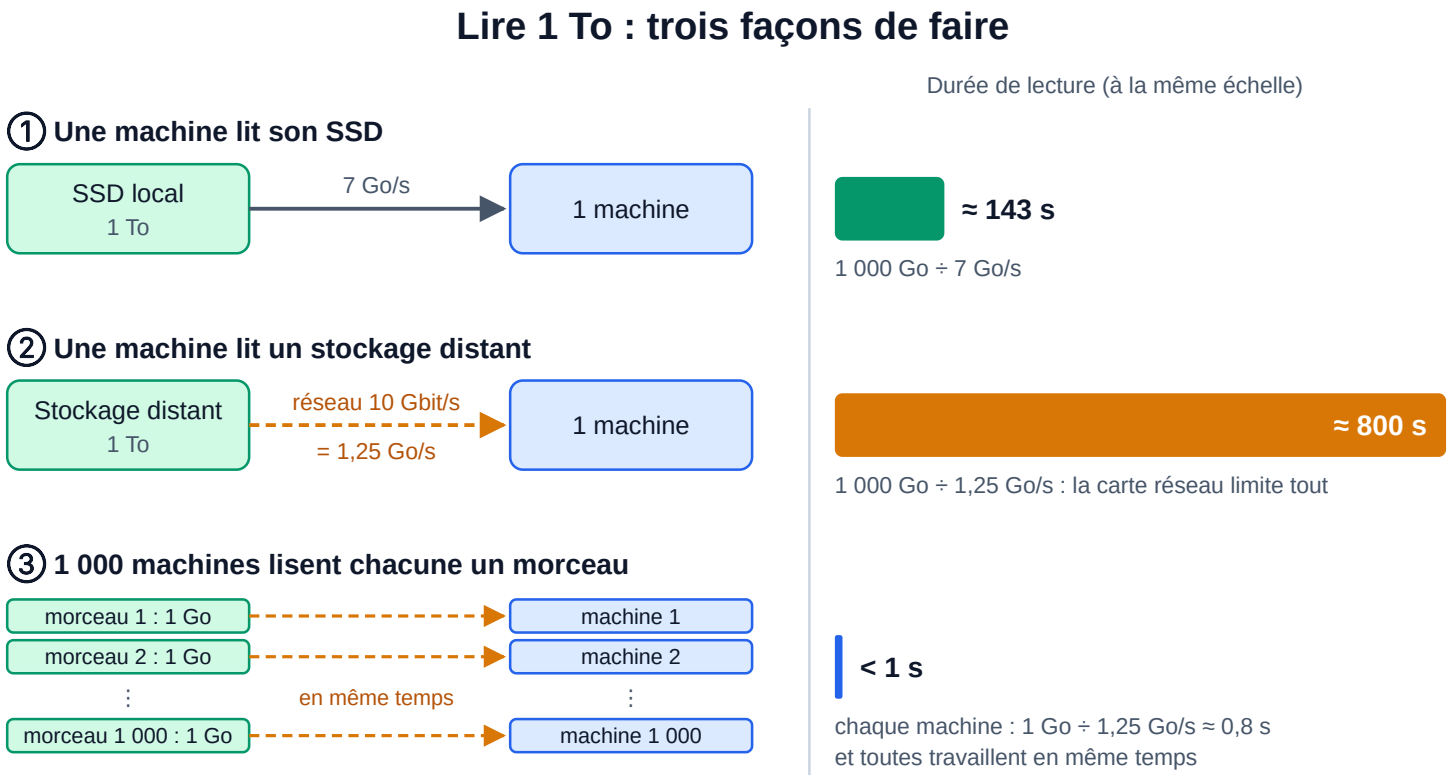


*Schéma : la même quantité de données (1 To = 1 000 Go) lue de trois façons

2.5 Ordres de grandeur à retenir (suite)

- Avec un SSD à 7 Go/s : environ 143 s.
- Avec une seule carte réseau à 10 Gbit/s (1,25 Go/s) : environ 800 s.
- Avec 1 000 machines qui lisent chacune 1 Go en parallèle : moins d'une seconde.

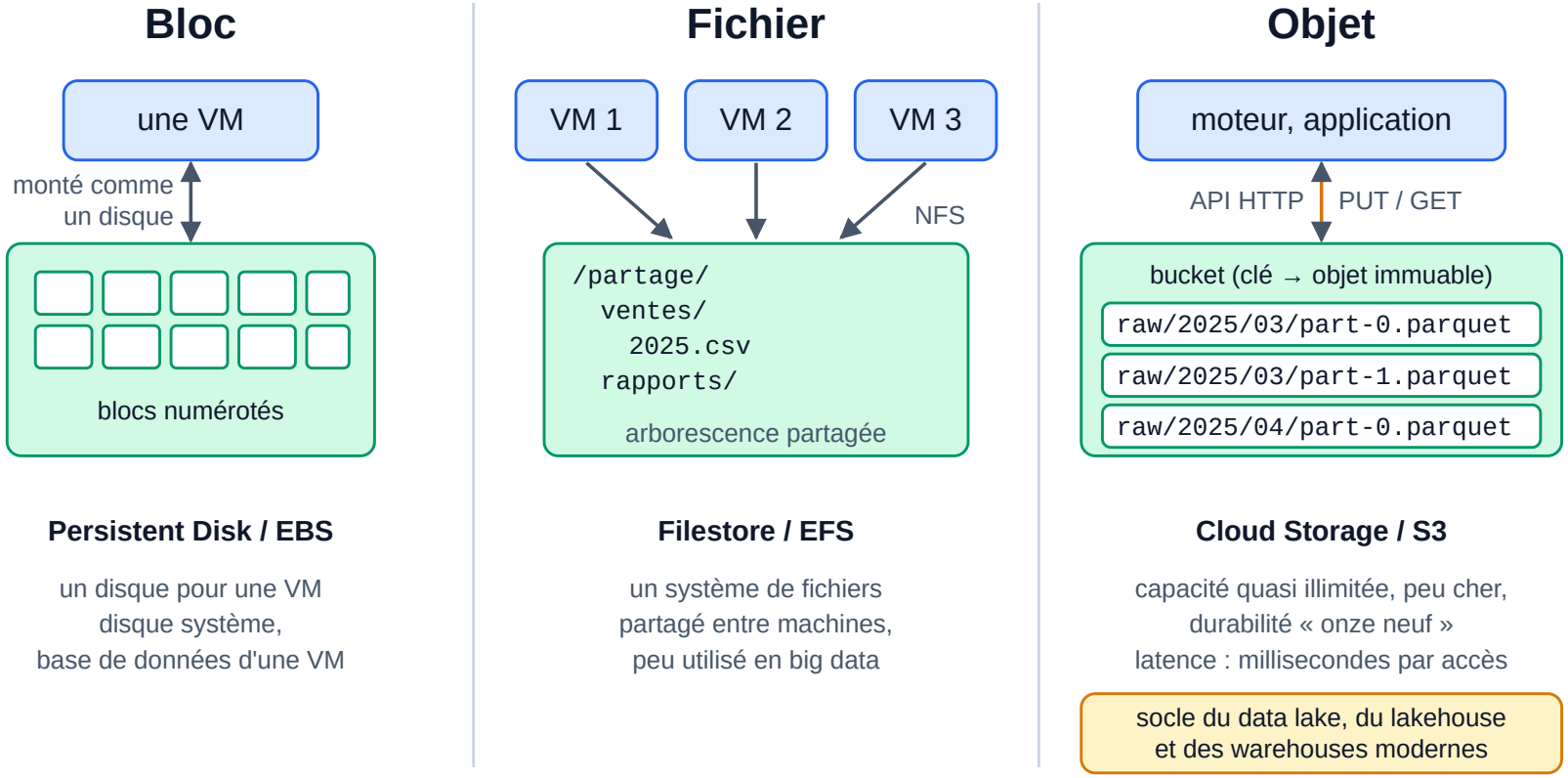
Conclusion : **seul le parallélisme massif permet d'interroger des téraoctets en quelques secondes**, et **chaque octet qu'on évite de lire est du temps et de l'argent gagnés**.

3. Les types de stockage dans le cloud

Type	Exemple GCP / AWS	Accès	Usage dans l'analytique
Bloc	Persistent Disk / EBS	Monté comme un disque sur une VM	Disque système, bases de données d'une VM
Fichier	Filestore / EFS	Système de fichiers partagé (NFS)	Partage entre machines, peu utilisé en big data
Objet	Cloud Storage / S3	API HTTP, objets immuables identifiés par une clé	**Socle du data lake, du lakehouse et du stockage des warehouses modernes**

- Le stockage objet offre une capacité quasi illimitée, un coût faible et une **durabilité annuelle de 99,999999999 %** (« onze neuf ») grâce à la redondance des données sur plusieurs machines et zones.
- Contrepartie : la latence d'un accès est élevée (millisecondes) et un objet ne se modifie pas en place, il se réécrit. Les moteurs lisent donc **peu de gros fichiers** plutôt que beaucoup de petits.
- La **durabilité** (ne pas perdre la donnée) s'obtient par réplication ou par codes correcteurs (*erasure coding*, ex. Reed-Solomon) ; la **disponibilité** (pouvoir la lire maintenant) est une notion distincte.

3. Les types de stockage dans le cloud (suite)



Persistent Disk / EBS

un disque pour une VM
disque système,
base de données d'une VM

Filestore / EFS

un système de fichiers
partagé entre machines,
peu utilisé en big data

Cloud Storage / S3

capacité quasi illimitée, peu cher,
durabilité « onze neuf »
latence : millisecondes par accès

socle du data lake, du lakehouse
et des warehouses modernes

Schéma : le bloc s'attache à une seule VM, le fichier se partage par NFS, l'objet se lit et s'écrit en entier par une API HTTP ; c'est ce dernier qui porte les data lakes et les warehouses.

4. Du hardware au moteur analytique

4.1 Deux familles d'usage : OLTP et OLAP

Avant de choisir comment ranger les données, il faut savoir **à quoi elles vont servir**. On distingue deux grandes familles de bases de données :

- **OLTP** (*OnLine Transaction Processing*, traitement de transactions en ligne) : la base qui **fait tourner l'application**. Elle enregistre chaque action au moment où elle se produit : une commande, un paiement, un changement d'adresse. Chaque requête touche **très peu de lignes**, mais il y en a énormément, et il faut répondre en quelques millisecondes. Exemple : la base PostgreSQL, MySQL.
- **OLAP** (*OnLine Analytical Processing*, traitement analytique en ligne) : la base qui sert à **analyser l'activité**. Elle répond à des questions sur de gros volumes d'historique. Chaque requête parcourt **des millions de lignes**, mais n'utilise que **quelques colonnes**. Exemple : un data warehouse comme BigQuery ou Redshift.

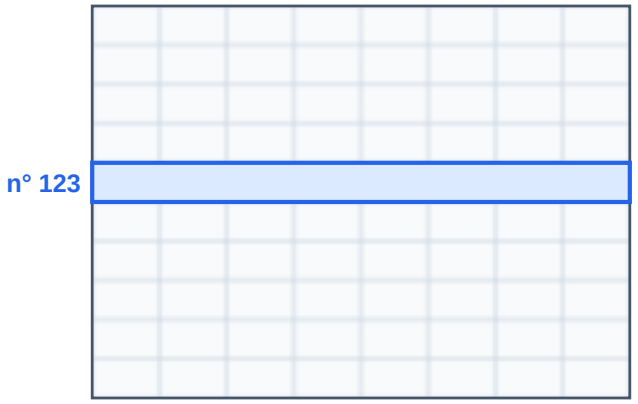
4.1 Deux familles d'usage : OLTP et OLAP (suite)

	OLTP	OLAP
Exemple de requête	« Authentification à un site web »	« Répartition des âges des utilisateurs d'un site web »
Lignes touchées	Quelques-unes	Des millions à des milliards
Colonnes touchées	Toutes	Quelques-unes
Écritures	Fréquentes, petites	Chargements en masse
Stockage	En lignes	En colonnes

4.1 Deux familles d'usage : OLTP et OLAP (suite)

OLTP (application)

« Enregistrer le paiement n° 123 »



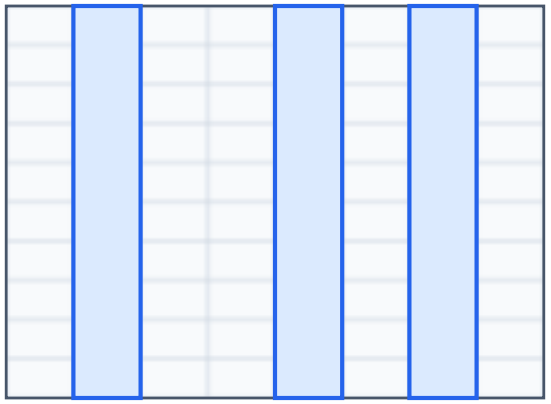
une ligne, toutes ses colonnes

Lignes touchées : quelques-unes
Écritures : fréquentes, petites

→ **stockage en lignes**

OLAP (data warehouse)

« Taux de fraude par pays et par mois sur 3 ans »



toutes les lignes, quelques colonnes (pays, date, fraude)

Lignes touchées : millions à milliards
Écritures : chargements en masse

→ **stockage en colonnes**

Schéma : l'OLTP lit une ligne et toutes ses colonnes, l'OLAP quelques colonnes sur toutes les lignes ; chaque profil appelle son mode de stockage.

4.1 Deux familles d'usage : OLTP et OLAP (suite)

À retenir : **l'OLTP lit et écrit des lignes entières, une à la fois ; l'OLAP lit quelques colonnes sur toutes les lignes**. Ces deux usages appellent deux façons différentes de ranger les données sur le disque : c'est l'objet de la partie suivante.

4.2 Stockage en lignes vs en colonnes

- **En lignes** (PostgreSQL, MySQL) : toutes les colonnes d'une ligne sont côte à côte. Idéal pour lire ou modifier **une commande entière** : c'est le choix des bases **OLTP**.
- **En colonnes** (BigQuery, Redshift, Snowflake, Parquet) : toutes les valeurs d'une colonne sont côte à côte. Une requête `SELECT SUM(amount) FROM payments` ne lit que la colonne `amount`, pas les 30 autres.
- Exemple ShopFlow : la table `payments` a 20 colonnes ; une requête qui en utilise 2 lit environ **10 fois moins de données** en colonnes.

4.2 Stockage en lignes vs en colonnes (suite)

`SELECT SUM(amount) FROM payments`

Table logique
(valeurs illustratives)

payment_id	customer_id	card_country	amount
1	42	FR	59.90
2	17	DE	120.00
3	42	FR	15.00
4	8	FR	230.00

En lignes (PostgreSQL, MySQL)
un bloc sur disque = des lignes entières

bloc 1	1	42	FR	**59.90**
bloc 2	2	17	DE	**120.00**
bloc 3	3	42	FR	**15.00**
bloc 4	4	8	FR	**230.00**

4 blocs lus sur 4
pour n'utiliser qu'une colonne

En colonnes (BigQuery, Redshift, Parquet)
un bloc sur disque = les valeurs d'une colonne

payment_id	1	2	3	4
customer_id	42	17	42	8
card_country	FR	DE	FR	FR
amount	**59.90**	**120.00**	**15.00**	**230.00**

1 bloc lu sur 4
les blocs gris ne sont pas lus

Schéma : pour additionner `amount`, le stockage en lignes lit tous les blocs, le stockage en colonnes un seul ; les blocs gris ne quittent jamais le disque.

4.3 Compression et encodage

- Les valeurs d'une même colonne se ressemblent (même type, souvent répétées), donc elles se compressent très bien : **dictionnaire** (`country = 'FR'` stocké comme un petit entier), **RLE** (*run-length encoding* : « FR » × 10 000), **bit-packing**.
- BigQuery (format Capacitor) va jusqu'à **réordonner les lignes** pour allonger les répétitions et améliorer le RLE.
- Moins d'octets à lire = moins de disque, moins de réseau, plus de données dans le cache CPU.

4.3 Compression et encodage (suite)

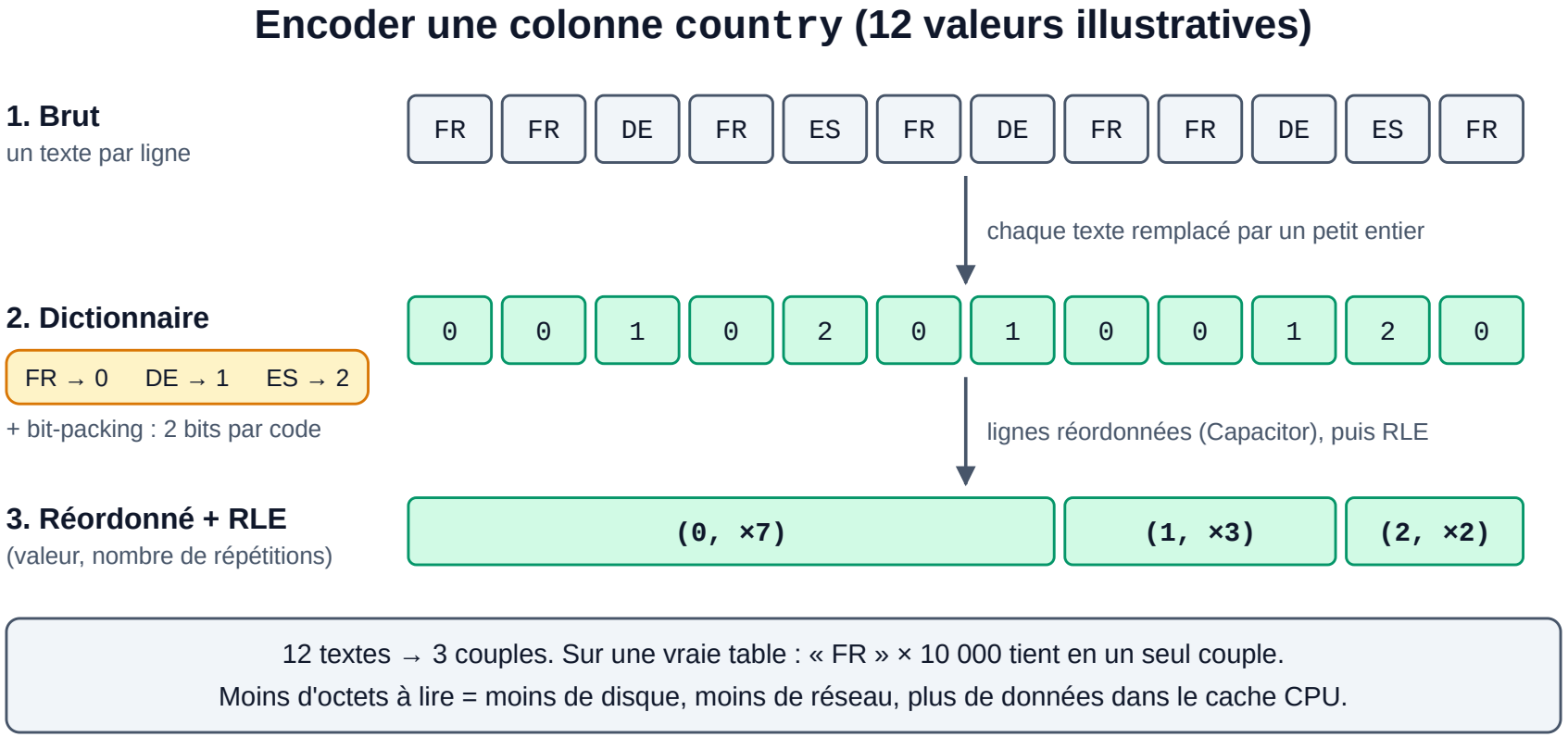


Schéma : le dictionnaire remplace chaque texte par un petit entier, puis le réordonnancement des lignes (comme dans BogQuery) regroupe les valeurs identiques pour que le RLE les résume en quelques couples.

4.4 Optimisations d'exécution : vectorisation, data skipping et parallélisme MPP

- **Exécution vectorisée** : un moteur classique traite les lignes **une par une** : pour multiplier par 2 les valeurs de 8 lignes, il exécute 8 fois l'instruction « multiplier par 2 ». Un moteur vectorisé charge les valeurs d'une colonne **par lots (vecteurs)** et le processeur applique la même opération **à tout le lot avec une seule instruction** : le même calcul demande alors 8 fois moins d'instructions. Les moteurs réels traitent des lots de plusieurs milliers de valeurs.
- **Data skipping** : chaque bloc de données stocke, pour chaque colonne, sa **valeur minimale et sa valeur maximale**. Avant de lire un bloc, le moteur compare le filtre de la requête à ces min/max : si le bloc ne peut pas contenir de réponse, il **n'est pas lu du tout**. C'est la base du partitionnement et du clustering (4.5).

4.4 Optimisations d'exécution : vectorisation, data skipping et parallélisme MPP (suite)

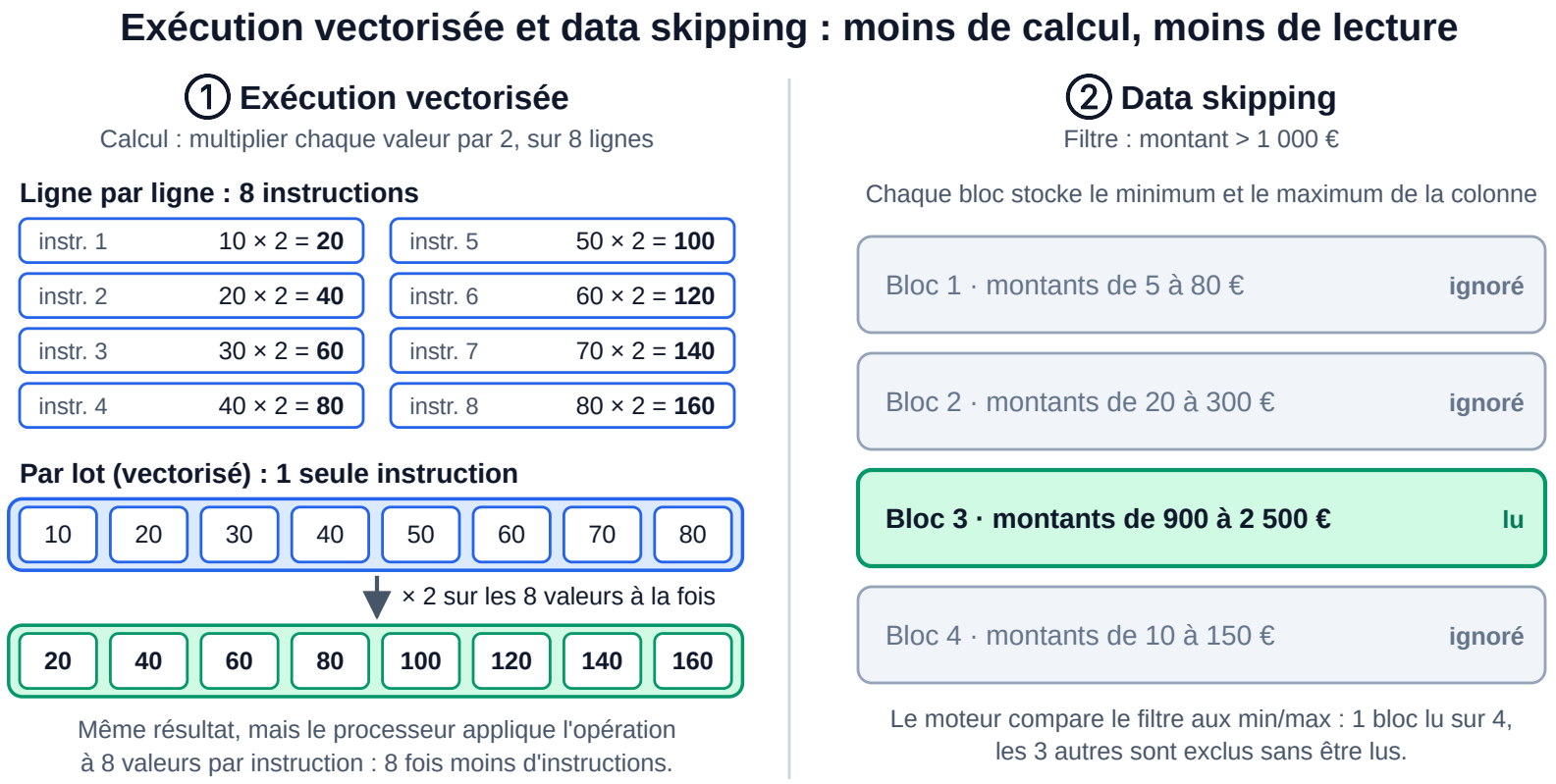


Schéma : à gauche, la même multiplication (× 2) faite en 8 instructions ligne par ligne, ou en 1 instruction sur un lot de 8 valeurs ; à droite, le moteur compare le filtre aux min/max de chaque bloc et exclut les blocs inutiles sans les lire.

4.4 Optimisations d'exécution : vectorisation, data skipping et parallélisme MPP (suite)

- **Parallélisme MPP** (*Massively Parallel Processing*, traitement massivement parallèle) : la table est découpée en morceaux répartis entre des centaines ou des milliers de machines. Chaque machine calcule un **résultat partiel** sur son morceau, **en même temps** que les autres ; un **coordinateur** assemble ensuite ces résultats partiels, qui sont tout petits. C'est le principe de l'exercice de 2.5, appliqué au calcul.

4.4 Optimisations d'exécution : vectorisation, data skipping et parallélisme MPP (suite)

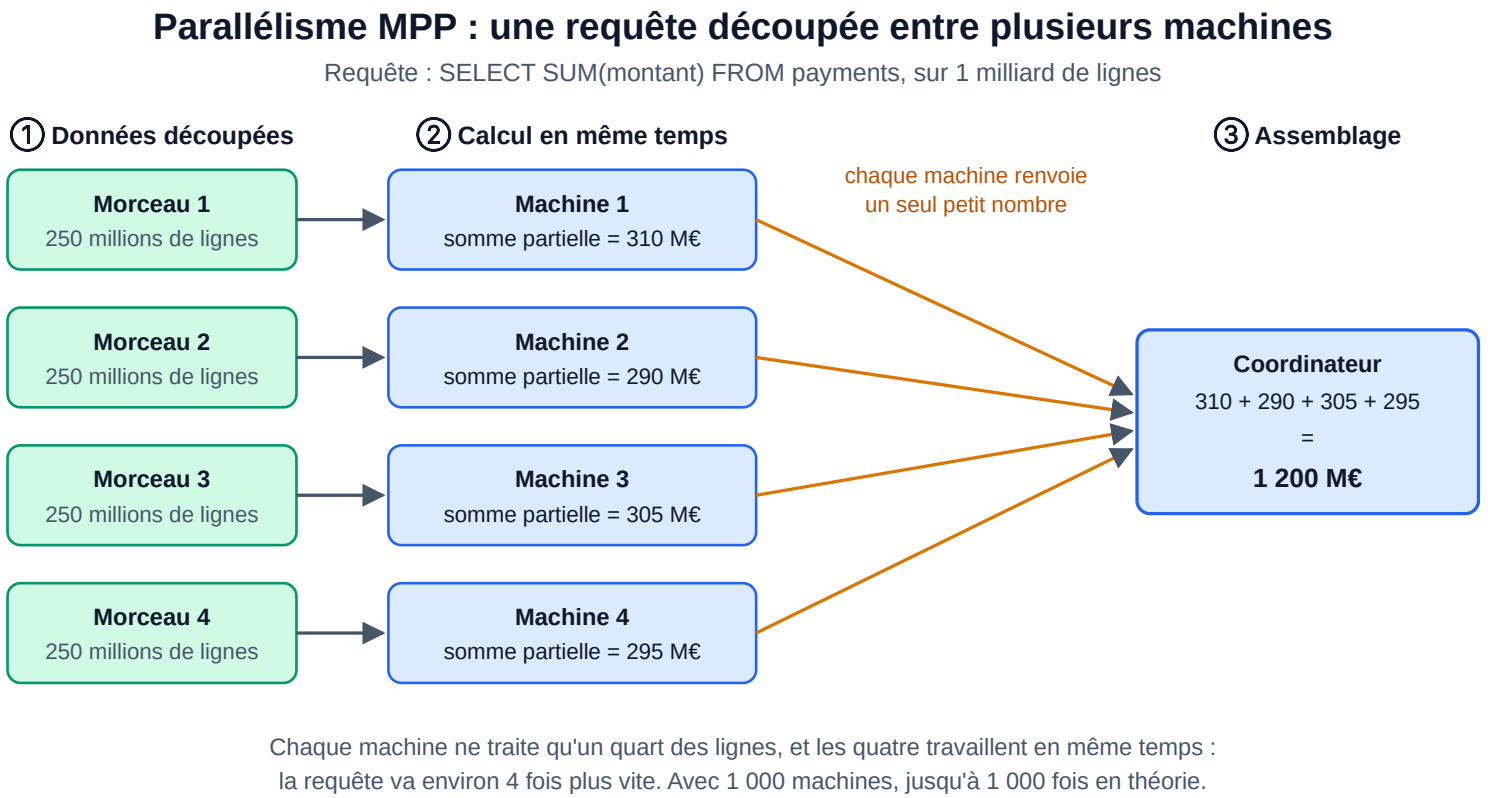


Schéma : chaque machine additionne seulement son quart des lignes, les quatre en même temps, puis le coordinateur additionne quatre nombres ; le temps de la requête est divisé à peu près par le nombre de machines.

4.5 Partitionnement et clustering : organiser les données pour lire moins

Le data skipping (4.4) n'est efficace que si les min/max des blocs sont **sélectifs**. Si les lignes sont rangées dans l'ordre d'arrivée, chaque bloc contient un peu de toutes les valeurs : ses min/max couvrent presque tout l'intervalle et n'excluent rien. Le partitionnement et le clustering organisent physiquement les données pour que le moteur puisse exclure un maximum de données.

Le partitionnement

- **Principe** : la table est **découpée physiquement** en segments de stockage séparés, les **partitions**, selon la valeur d'une **colonne de partitionnement**. Chaque partition est stockée à part (dans BigQuery, un segment interne ; dans un data lake, souvent un répertoire par valeur, par exemple `date=2026-03-01/`).
- **Colonne choisie** : le plus souvent une **date ou un horodatage**, découpé par heure, jour, mois ou année. BigQuery accepte aussi une plage d'entiers, ou l'heure d'ingestion des données.
- **Élagage des partitions** (*partition pruning*) : la liste des partitions est connue dans les **métadonnées** de la table. Au moment de **planifier** la requête, avant toute lecture, le moteur compare le filtre (`WHERE payment_date BETWEEN '2026-03-01' AND '2026-03-31'`) à cette liste et écarte les partitions inutiles. Condition : le filtre doit porter **directement sur la colonne de partitionnement**.
- **Limite** : chaque partition a un coût de gestion (métadonnées, petits fichiers). Trop de partitions trop petites ralentissent la table au lieu de l'accélérer : on choisit une granularité adaptée au volume.

Le partitionnement (suite)

Partitionner : un tiroir par mois

Table payments · question : « les paiements de mars ? »

Sans partitionnement

les mois sont mélangés dans tous les blocs

janv., mai, mars	avr., mars, juin	févr., mars, janv.
mai, juin, mars	mars, avr., févr.	juin, janv., mars
févr., mai, mars	avr., mars, janv.	mars, juin, mai

9 blocs sur 9 lus : il y a du mars partout

Partitionnée par mois

chaque mois est rangé dans son propre tiroir

Tiroir janv. jamais ouvert	**Tiroir févr.** jamais ouvert
Tiroir mars ouvert et lu	**Tiroir avr.** jamais ouvert
Tiroir mai jamais ouvert	**Tiroir juin** jamais ouvert

1 tiroir sur 6 lu : les autres sont ignorés

Le filtre sur la colonne de partition (ici la date) dit au moteur quels tiroirs ouvrir avant même de lire.

Schéma : sans partitionnement, il y a du mars dans tous les blocs et tout est lu ; partitionnée par mois, la requête ne lit que la partition de mars.

Le clustering

- **Principe** : les lignes sont **triées** selon une ou plusieurs **colonnes de clustering** (jusqu'à 4 dans BigQuery et Databricks), puis rangées dans les blocs dans cet ordre. Les valeurs proches se retrouvent dans les mêmes blocs, donc les **min/max de chaque bloc deviennent étroits**.
- **Élagage des blocs** : pendant l'exécution, le moteur utilise ces min/max (le data skipping de 4.4) pour ne lire que les blocs qui peuvent contenir la réponse.
- **Ordre des colonnes** : dans **BigQuery**, les lignes sont triées d'abord sur la première colonne, puis sur la deuxième à valeur égale, etc. Un filtre sur la **première colonne** de clustering y est donc le plus efficace. Le *liquid clustering* de **Databricks**, lui, ne donne la priorité à aucune clé : on peut les déclarer **dans n'importe quel ordre**.
- **Maintenance** : les nouvelles lignes arrivent dans le désordre. Le moteur doit retrier régulièrement : BigQuery le fait **automatiquement en arrière-plan** ; dans Databricks, la commande `OPTIMIZE` regroupe et retrie les fichiers.

Le clustering (suite)

- **Avec le partitionnement** : dans BigQuery, on partitionne d'abord (par date), puis on clusterise **à l'intérieur de chaque partition**. Databricks recommande au contraire le *liquid clustering* seul pour les nouvelles tables, sans partitionnement, sauf pour les très grosses tables.

Le clustering (suite)



Schéma : sans tri, la France peut se trouver dans chaque bloc ; triées par pays, ses lignes sont regroupées dans un seul bloc, et les autres blocs sont exclus par leurs min/max.

Le clustering (suite)

	Partitionnement	Clustering
Organisation physique	**Découpage** en segments séparés	**Tri** des lignes à l'intérieur des blocs
Colonnes	Une seule, souvent une date	Jusqu'à 4 (l'ordre compte dans BigQuery, pas dans Databricks)
Ce qui est exclu	Des partitions entières, **à la planification**	Des blocs, **à l'exécution** grâce aux min/max
Point d'attention	Pas trop de petites partitions	Retri régulier des nouvelles données

Selon la plateforme :

- **BigQuery** : `PARTITION BY` (date, horodatage, plage d'entiers ou heure d'ingestion) et `CLUSTER BY` (jusqu'à 4 colonnes) à la création de la table.
- **Databricks** : *liquid clustering* (`CLUSTER BY`, jusqu'à 4 clés), recommandé pour toutes les nouvelles tables ; il ne se combine pas avec le partitionnement, réservé aux très grosses tables.
- **Redshift** : les **clés de tri** (*sort keys*) jouent le rôle du clustering, avec des min/max par bloc.

Vous mesurerez l'effet du partitionnement et du clustering en octets lus dans la partie 3 du TP.

Sources : Le clustering

Sources : Google Cloud, Tables partitionnées et Tables clusterisées ; Databricks, Liquid clustering et Quand partitionner une table ; AWS, Clés de tri Redshift (min/max par bloc de 1 Mo).

5. Qu'est-ce qu'un data warehouse ?

- **Définition (Inmon)** : une collection de données **orientée sujet, intégrée, historisée et non volatile**, conçue pour l'aide à la décision.
- **Un data warehouse est une base OLAP** (voir 4.1) : il sert à analyser l'historique, pas à faire tourner l'application au quotidien.
- **ETL vs ELT** : autrefois on transformait avant de charger (ETL) car le calcul du warehouse était rare et cher. Avec un calcul élastique dans le cloud, on charge brut puis on transforme en SQL dans le warehouse (ELT).

6. Le parcours d'une requête dans un data warehouse

Tous les data warehouses cloud (BigQuery, Redshift, Snowflake, Databricks SQL…) ont des noms et des composants différents, mais **une requête y suit toujours le même chemin**. On le décrit ici de façon générale, avec la requête ShopFlow suivante :

```sql
SELECT card_country, SUM(amount) AS montant
FROM payments
WHERE payment_ts >= '2026-01-01'
GROUP BY card_country;
```

6. Le parcours d'une requête dans un data warehouse (suite)

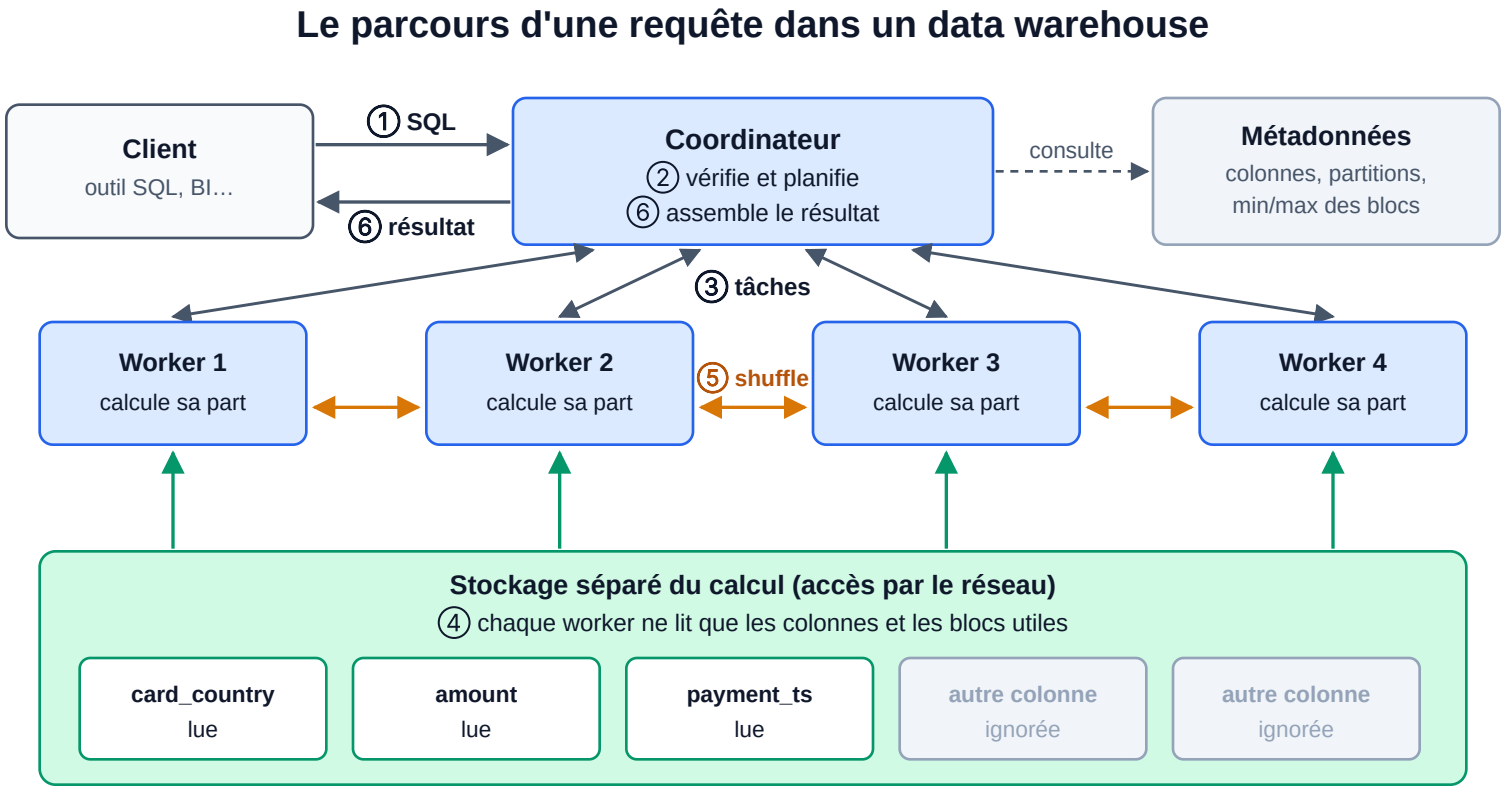


Schéma : la requête descend du client vers les workers (étapes 1 à 3), les workers lisent et calculent en parallèle (4 et 5), puis les petits résultats remontent jusqu'au client (6).

6. Le parcours d'une requête dans un data warehouse (suite)

1. **Envoi** : l'utilisateur (ou un outil de BI) envoie le texte SQL au data warehouse. Un **coordinateur** le reçoit : c'est le point d'entrée unique.
2. **Planification** : le coordinateur vérifie la requête (syntaxe, droits d'accès) puis consulte les **métadonnées** de la table : quelles colonnes existent, comment la table est partitionnée, quels min/max a chaque bloc. Un **optimiseur** en déduit un **plan d'exécution**, c'est-à-dire la liste des étapes à faire et dans quel ordre. C'est ici que le moteur **élimine les partitions inutiles** (4.5) : si la table est partitionnée par date, seules les partitions de 2026 seront lues.
3. **Distribution** : le plan est découpé en petites tâches, envoyées à de nombreux **workers** (des machines de calcul) qui travaillent en même temps (MPP, 4.4).

6. Le parcours d'une requête dans un data warehouse (suite)

4. **Lecture** : chaque worker lit **seulement les colonnes utiles** (`card_country`, `amount`, `payment_ts`) et **seulement les blocs utiles** (data skipping, 4.4) depuis le stockage, qui est séparé du calcul et accessible par le réseau (2.4). Les données souvent lues peuvent déjà se trouver dans un cache en RAM ou sur SSD local (2.3).
5. **Calcul et échange** : chaque worker filtre et calcule une somme partielle par pays sur son morceau. Pour regrouper un même pays, les workers s'**échangent** ensuite leurs résultats intermédiaires par le réseau (le ***shuffle***) : c'est l'étape la plus coûteuse quand il y a des jointures ou des `GROUP BY` sur beaucoup de lignes.
6. **Assemblage** : le coordinateur rassemble les résultats partiels, qui sont tout petits (une ligne par pays), et renvoie le résultat au client.

À retenir : le travail coûteux se fait aux étapes 4 et 5. Chaque octet qu'on évite de lire (étapes 2 et 4) ou de faire circuler (étape 5) rend la requête plus rapide et moins chère.

6. Le parcours d'une requête dans un data warehouse (suite)

Comment on paie : selon l'offre, on paie soit **le volume de données lu** par la requête (par exemple BigQuery en mode « à la demande » : 6,25 $ par Tio lu, le premier Tio de chaque mois étant gratuit), soit **le temps de calcul** des workers réservés. Dans les deux cas, lire moins coûte moins.

7. Conséquences pour écrire des requêtes

Points communs à retenir : **stockage en colonnes**, **séparation du stockage et du calcul**, **exécution massivement parallèle**, et **un cache rapide** (RAM ou SSD) entre les deux.

Règles d'écriture de requêtes qui en découlent (mises en pratique dans le TP) :

1. Ne lire que les colonnes nécessaires (pas de `SELECT *`).
2. Filtrer sur la colonne de partition ou de tri pour que le moteur saute des blocs.
3. Filtrer et agréger **avant** de joindre, pour réduire ce qui passe dans le shuffle.
4. Éviter les jointures qui multiplient les lignes.
5. Estimer le coût avant d'exécuter (*dry run* dans BigQuery, `EXPLAIN` dans la plupart des moteurs).

Sources : 7. Conséquences pour écrire des requêtes

Sources : Google Cloud, bonnes pratiques de performance BigQuery.